



Sistemas Distribuidos I (75.74)

Tiempo, Relojes, Orden y Cortes de Estado

Tiempo. Relojes. Orden de Mensajes.
Cortes de Estado

Docentes

- | | | |
|---------------------|-----------------------------|-------------------|
| ■ Pablo Roca | ■ Manuel Reberendo | ■ W. Gabriel Diem |
| ■ Franco Barreneche | ■ Máximo Gismondi | ■ Máximo Utrera |
| ■ Nicolás Zulaica | ■ Camila Sebellin | ■ Ivan Pfaab |
| ■ Franco Papa | ■ Patricio Tourne Passarino | |



- **Tiempo y relojes - Relojes Físicos**

- Tiempo y relojes - Relojes Lógicos

- Sincronismo

- Orden de Mensajes

- Estado y consistencia



Magnitud para medir duración y separación de eventos.



Se lo puede definir mediante una variable monotónica creciente:

- En sistemas de computación será discreta
- No necesariamente vinculada con la hora de la vida real

El tiempo siempre avanza, nunca retrocede.



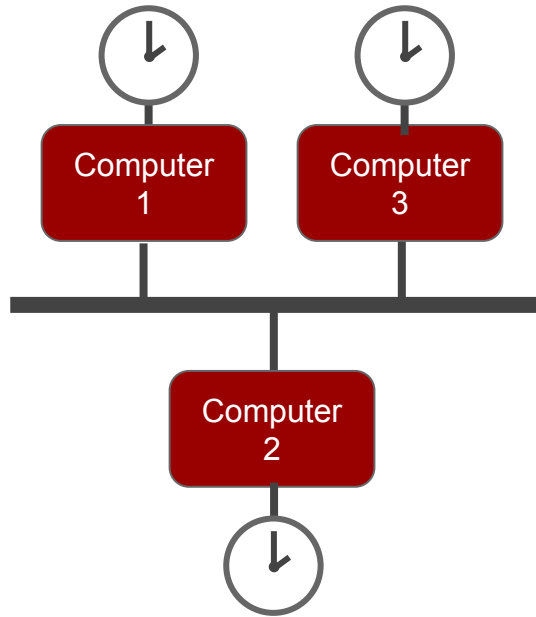
La medición del tiempo permite:

- Ordenar y sincronizar
- Marcar la ocurrencia de un suceso
 - **Timestamps:** puntos absoluto en la línea de tiempo
- Contabilizar duración entre sucesos
 - **Timespans:** intervalo en la línea de tiempo

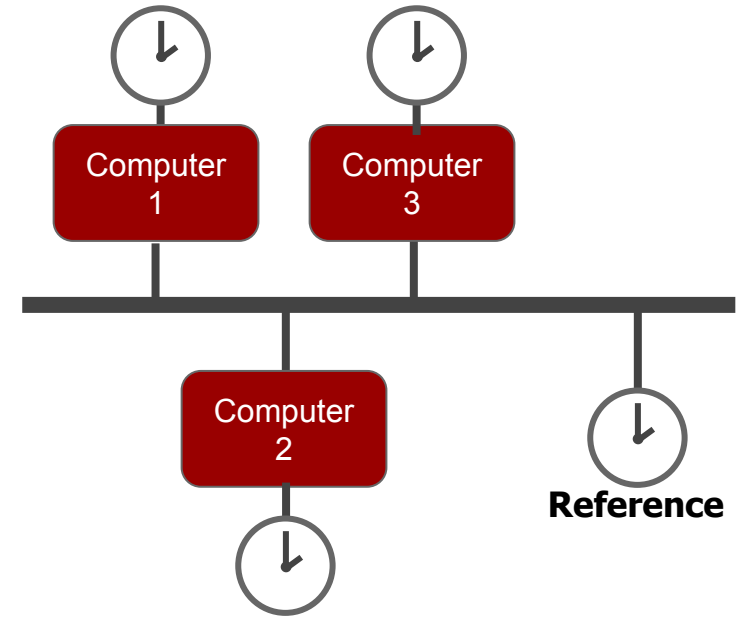
Relojes Físicos | Locales vs Globales



Locales



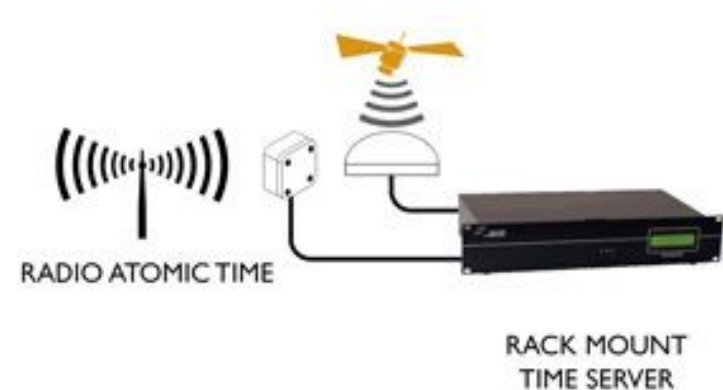
Locales + Global



Relojes Físicos



- Brindan la fecha y hora del día
- Pueden ser de cuarzo, atómicos, por GPS
- Los cambios de temperatura, presión y humedad descalibran relojes: *drift*





Relojes Físicos | Referencias Globales

GMT (Greenwich Mean Time)

- Basado en el tiempo de rot. Terrestre (t. astronómico)

UTC (Universal Time Coordinated)

- Basado en medición de relojes atómicos
- Dif. de +/-1 seg con GMT y recibe ajustes periódicos

GPS time (Global Positioning System time)

- Basado en relojes atómicos en la Tierra
- No recibe ajustes pero permite ajustar satélites

TAI (Temps Atomique International)

- 200 relojes atómicos en 70 países
- No recibe ajustes astronómicos

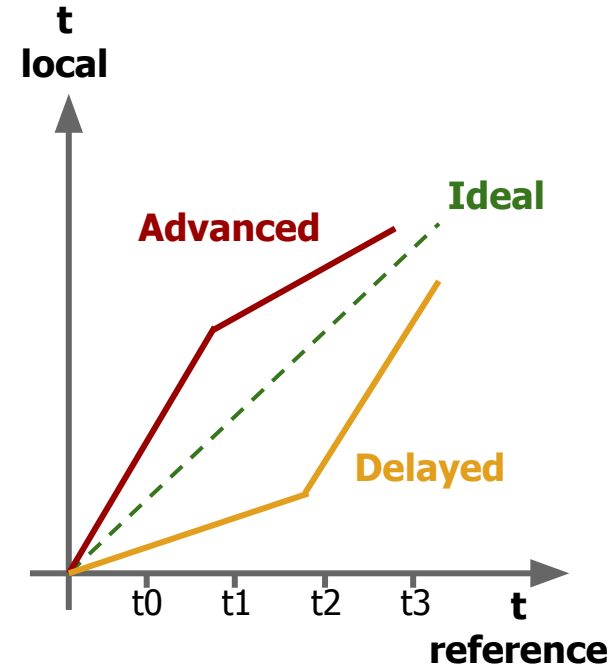


Source: wikipedia.org



Relojes Físicos | *Drift*

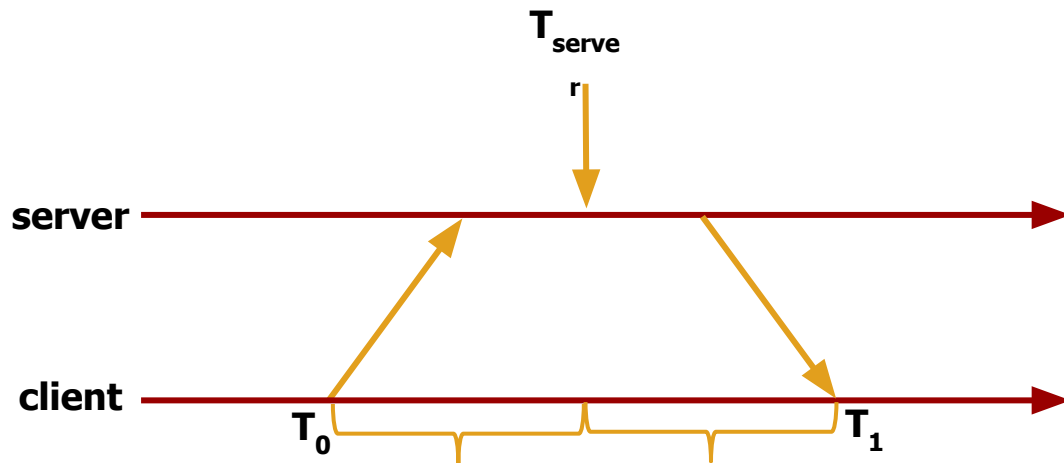
- Los relojes físicos no son confiables para su comparación por efecto del *drift*
- Hay que sincronizarlos periódicamente:
 - Medir el desvío respecto de un reloj de referencia (UTC, GPS, etc.)
 - Aplicar corrección o compensación lineal cambiando la frecuencia del reloj local
 - Nunca atrasar un reloj
- Hay que sincronizarlos al despertar la computadora





Relojes Físicos | *Drift - Algoritmo de Cristian*

- Realiza una compensación del delay existente al obtener la medida de tiempo
 - T_0 : Cliente envía request
 - T_1 : Cliente recibe respuesta
 - Hipótesis: *Delays de red constantes & Sin tiempo de procesamiento*



$$T_{\text{new}} = T_{\text{server}} + (T_1 - T_0) / 2$$

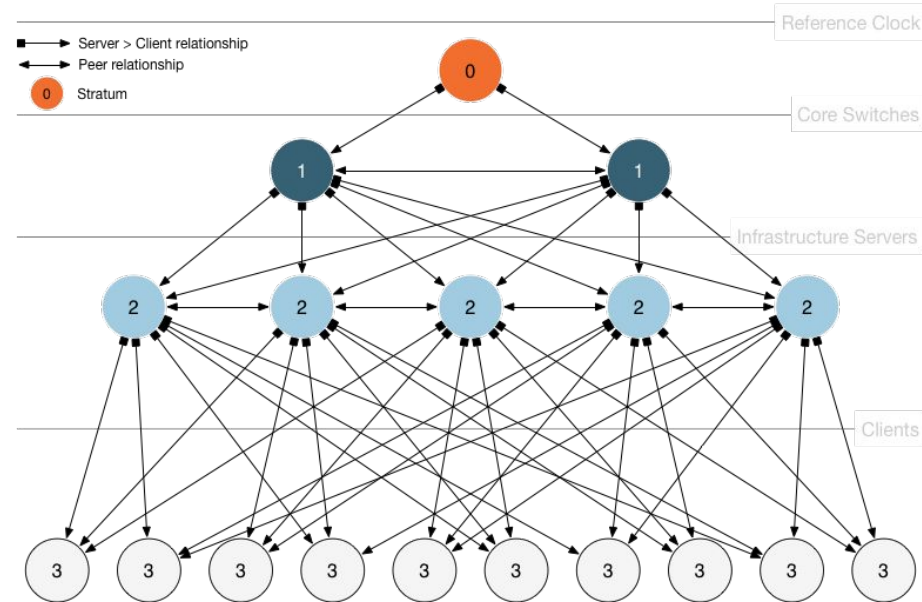


- **Sincronización**
 - Clientes (UTC) sincronizados aunque existan delays en la red
 - Análisis estadístico para filtrar data y obtener resultados de calidad
- **Alta disponibilidad**
 - Sobrevivir a largas caídas de conectividad
 - Rutas redundantes
 - Servidores redundantes
- **Escalabilidad**
 - Gran número de clientes sincronizados de forma frecuente
 - Debe tener en cuenta efectos de drift



Basado en stratus (estratos)

- Estrato 0: Master clocks
- Estrato 1: Servidores conectados directamente a master clocks
- Estrato 2: Servidores sincronizados con servidores en estrato 1
- Estrato N: Servidores sincronizados con servidores en estrato N-1
- Servidores sincronizados entre sí con conexiones peer-to-peer
- Mensajes enviados de forma no *reliable* (UDP puerto 123)





- **Modo multicast/broadcast**
 - Usado en LANs de alta velocidad
 - Eficiente pero de baja precisión
- **Modo Cliente-Servidor (RPC)**
 - Grupos de aplicaciones se conectan formando un grupo
 - Aplicaciones entre sí no pueden sincronizarse
- **Modo Simétrico (Peer Mode)**
 - Peers sincronizados entre sí para proveer backup mutuo
 - Utilizado en estratos 1 y 2



- ☐ Tiempo y relojes - Relojes Físicos
- ☒ **Tiempo y relojes - Relojes Lógicos**
- ☐ Sincronismo
- ☐ Orden de Mensajes
- ☐ Estado y consistencia



Evento

- Suceso relativo al proceso P_i que modifica su estado

Estado

- Valores de todas las variables del proceso P_i en un momento dado

Relación 'ocurre antes' (*happened before*) :

- Relación de causalidad entre eventos o estados tales que:
 - $a \rightarrow b$, si a, b pertenecen al mismo proceso P_i y a ocurre antes de b
 - $a \rightarrow b$, si a es un evento de P_i , b es un evento de P_j , a es el envío del mensaje ' m ' a P_j y b es la recepción del mensaje ' m ' desde P_i
 - $a \rightarrow c$, si $a \rightarrow b$ y $b \rightarrow c$ (transitividad)

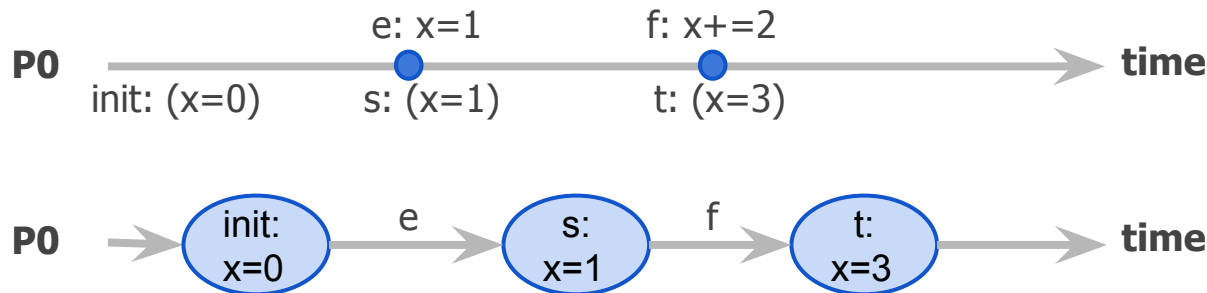


Relojes Lógicos | Definición

Dado S , el conjunto de todos los estados locales posibles del sistema y $\rightarrow$ la relación temporal de implicancia 'ocurre antes' (o *happened before*), un reloj lógico es una función C monotónica creciente que mapea estados con un número Natural y garantiza:

$$\forall s, t \in S : s \rightarrow t \Rightarrow C(s) < C(t)$$

Ej.:



$$s \rightarrow t$$
$$C(s) < C(t)$$



Relojes Lógicos | Algoritmo de Lamport

Pi:

var:

$c := 0$

send event (s, send, t):

//include t.c in msg

//send msg

$t.c := s.c + 1$

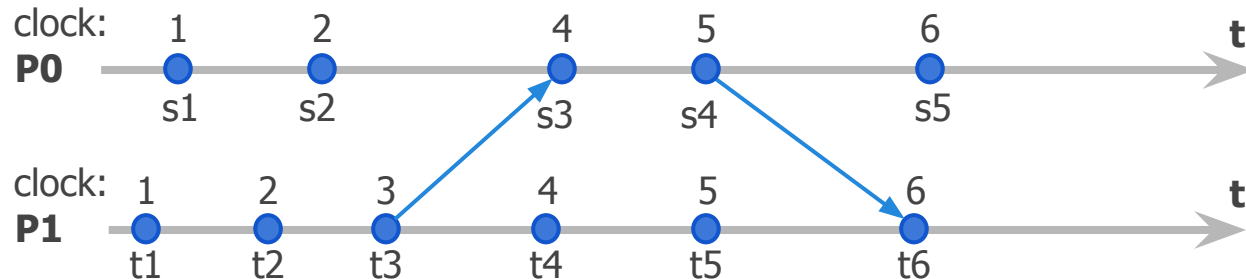
receive event (s, receive(u), t):

//receive msg from u

$t.c := \max(s.c, u.c) + 1$

internal event (s, internal, t):

$t.c := s.c + 1$





Relojes Lógicos | Inconvenientes

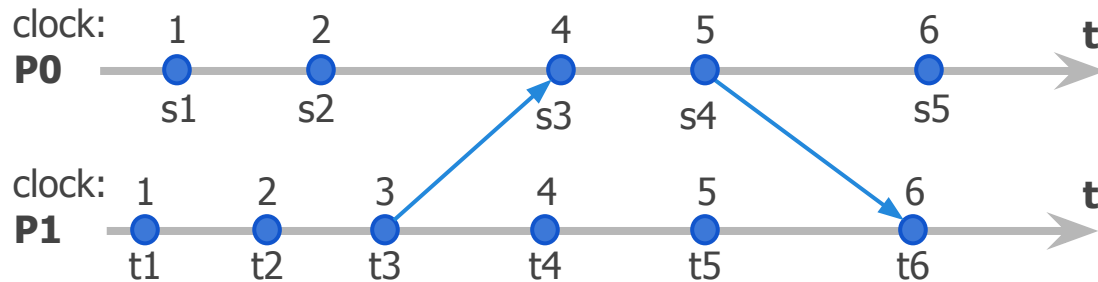
Los relojes de Lamport garantizan:

$$s \rightarrow t \Rightarrow C(s) < C(t)$$

pero:

$$C(s) < C(t) \not\Rightarrow s \rightarrow t$$

Ej.:



$$C(t1) < C(s4)$$
$$t1 \rightarrow s4$$

$$C(s1) < C(t4)$$
$$s1 \not\rightarrow t4$$

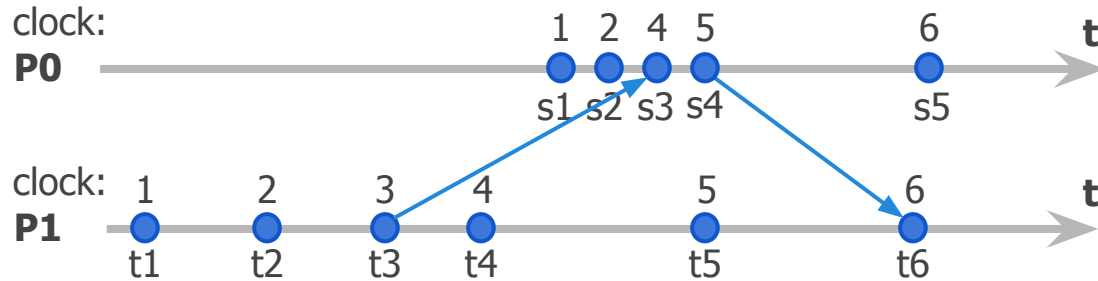


Relojes Lógicos | Inconvenientes (II)

El reloj de Lamport captura muy poca información sobre dos eventos:

1. progreso dentro de un proceso
2. progreso durante envíos de mensajes entre procesos

pero se pierde la información de qué evento ha ocurrido al almacenar un único escalár.





Relojes Lógicos | Corolario

Conociendo los relojes de Lamport se pueden detectar eventos paralelos

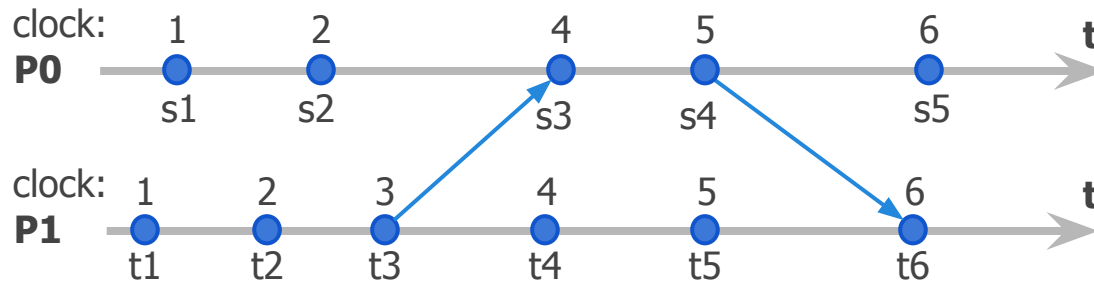
Ej:

- $c(s_2) = 2$
- $c(t_2) = 2$

$$C(s_2) \not\prec C(t_2) \Rightarrow s_2 \nrightarrow t_2$$

$$C(t_2) \not\prec C(s_2) \Rightarrow t_2 \nrightarrow s_2$$

$$s_2 \parallel t_2$$





Vector de Relojes | Definición

Un vector de relojes es el mapeo de todo estado del sistema compuesto por k procesos, con un vector de k números Naturales y garantiza:

$$\forall s, t \in S : s \rightarrow t \Leftrightarrow s.v < t.v$$

donde $s.v$ y $t.v$ son los vectores de k componentes para los estados s y t respectivamente y

$$s.v < t.v \Leftrightarrow \forall k : s.v[k] \leq t.v[k] \wedge \\ \exists j : s.v[j] < t.v[j]$$



Vector de Relojes | Implementación

Pi:

var:

$v[i] := 1$

$v[j] := 0$, con $i \neq j$

send event (s, send, t):

//include t.v in msg and send

$t.v := s.v$

$t.v[i] := t.v[i] + 1$

receive event (s, receive(u), t):

//receive msg from u

for $j := 1$ to N :

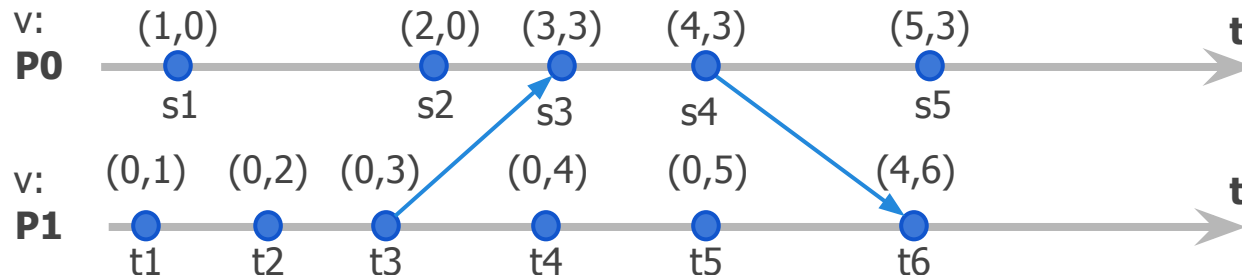
$t.v[j] := \max(s.v[j], u.v[j])$

$t.v[i] := t.v[i] + 1$

internal event (s, internal, t):

$t.v := s.v$

$t.v[i] := t.v[i] + 1$





Vector de Relojes | Corolario

t1 vs s4:

$$V(t1) < V(s4)$$

$$(0, 1) < (4, 3)$$

$$i=0 \Rightarrow \text{¿}0 < 4\text{?}$$

$$i=1 \Rightarrow \text{¿}1 < 3\text{?}$$

=> t1 -> s4

t4 vs s1:

$$V(t4) \text{ ¿<? } V(s1)$$

$$(0, 4) \text{ ¿<? } (1, 0)$$

$$i=0 \Rightarrow \text{¿}0 < 1\text{?}$$

$$i=1 \Rightarrow \text{¿}4 < 0\text{?}$$

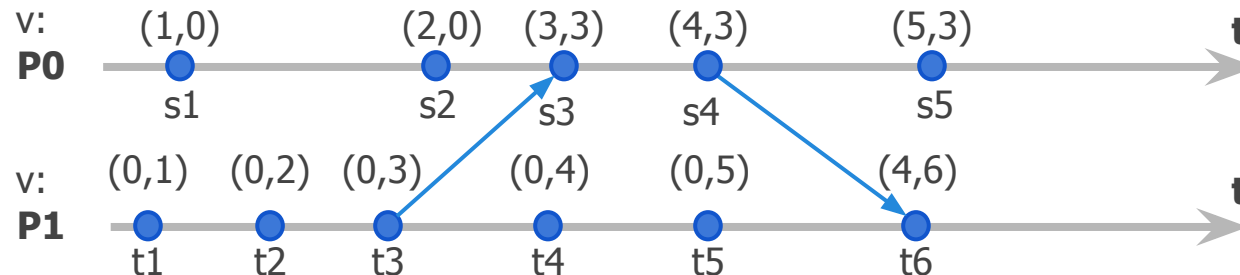
=> t4 || s1

$$V(s1) \text{ ¿<? } V(t4)$$

$$(1, 0) \text{ ¿<? } (0, 4)$$

$$i=0 \Rightarrow \text{¿}1 < 0\text{?}$$

$$i=1 \Rightarrow \text{¿}0 < 4\text{?}$$





- Tiempo y relojes - Relojes Físicos
- Tiempo y relojes - Relojes Lógicos
- **Sincronismo**
- Orden de Mensajes
- Estado y consistencia



Los términos sincrónico / asincrónico **dependen del contexto:**

- ***Ejecución de Eventos***
 - Sincrónico = bloqueante
 - Asincrónico = no bloqueante
- **Comunicación entre grupos**
 - Sincrónico = Entidades interactúan entre sí
 - Asincrónico = Entidades son independientes
- **Sistemas Digitales**
 - Sincrónico = Entidades coordinadas por el mismo reloj
 - Asincrónico = Entidades coordinadas por diferentes relojes
- **Sistemas Distribuidos**
 - ??



- Un algoritmo / protocolo es **sincrónico** si sus acciones pueden ser delimitadas en el tiempo
 - **Sincrónico**: Entrega de un mensaje posee un *timeout* conocido
 - **Parcialmente sincronico**: Entrega de un mensaje no posee un timeout conocido o bien el mismo es variable
 - **Asincrónico**: Entrega de un mensaje no posee *timeout* asociado
- ¿Cómo se generaliza esto para un sistema?
 - *Steadiness*
 - *Tightness*

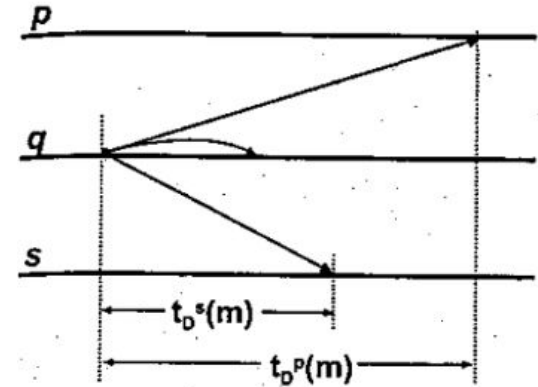


Tiempo de *Delivery* ($t_D^p(m)$)

- Es el tiempo que tarda un mensaje **m** en ser recibido luego de haber sido enviado hacia **p**

Timeout de *Delivery* (T_{Dmax})

- Todo mensaje enviado va a ser recibido antes de un tiempo T_{Dmax} conocido





Steadiness (σ)

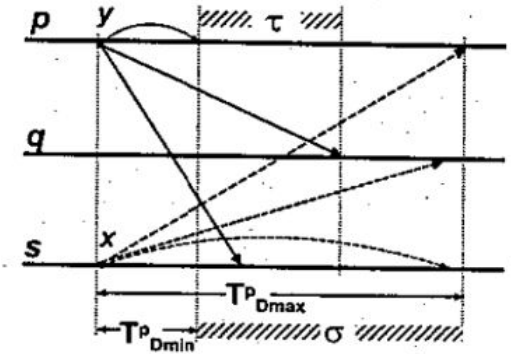
- Máxima diferencia entre el mínimo y máximo tiempo de delivery de cualquier mensaje recibido por un proceso

$$\sigma = \max(T_{Dmax} - T_{Dmin})$$

Tightness (τ)

- Máxima diferencia entre los tiempos de delivery para cualquier mensaje m

$$\forall p, q: \tau = \max_{m, p, q} (t_D^p(m) - t_D^q(m))$$



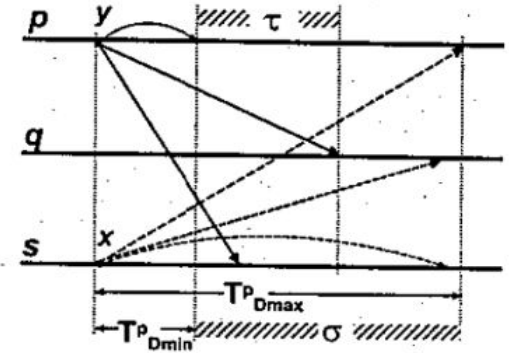


Steadiness (σ)

- Define la varianza con la cual un proceso observa que recibe los mensajes
- Define que tan constante (*steady*) es la Recepción de mensajes

Tightness (τ)

- Define la simultaneidad con la cual un mensaje es recibido por múltiples procesos



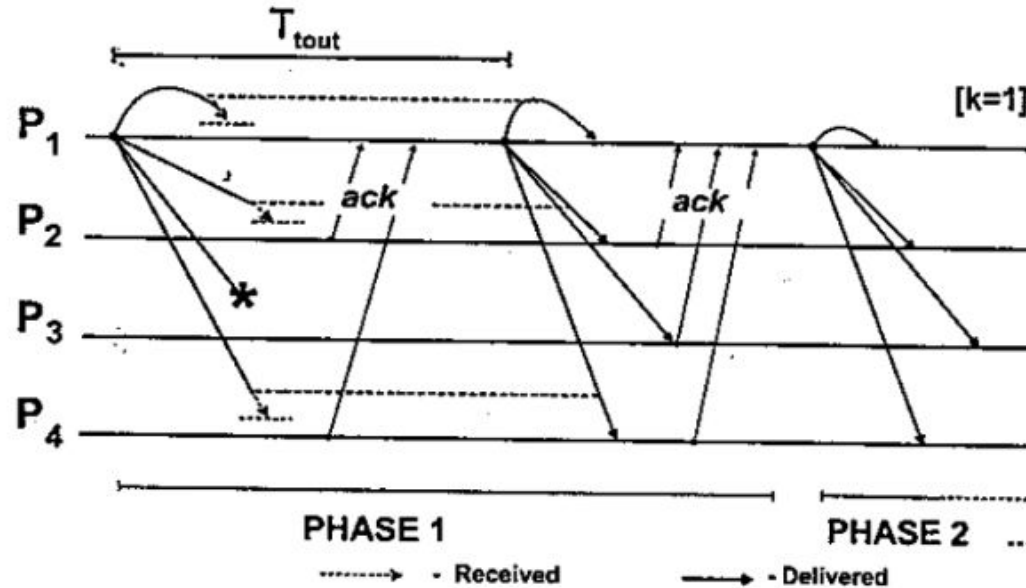


Figure 2.10. Unsteady and Untight Synchronous Protocol

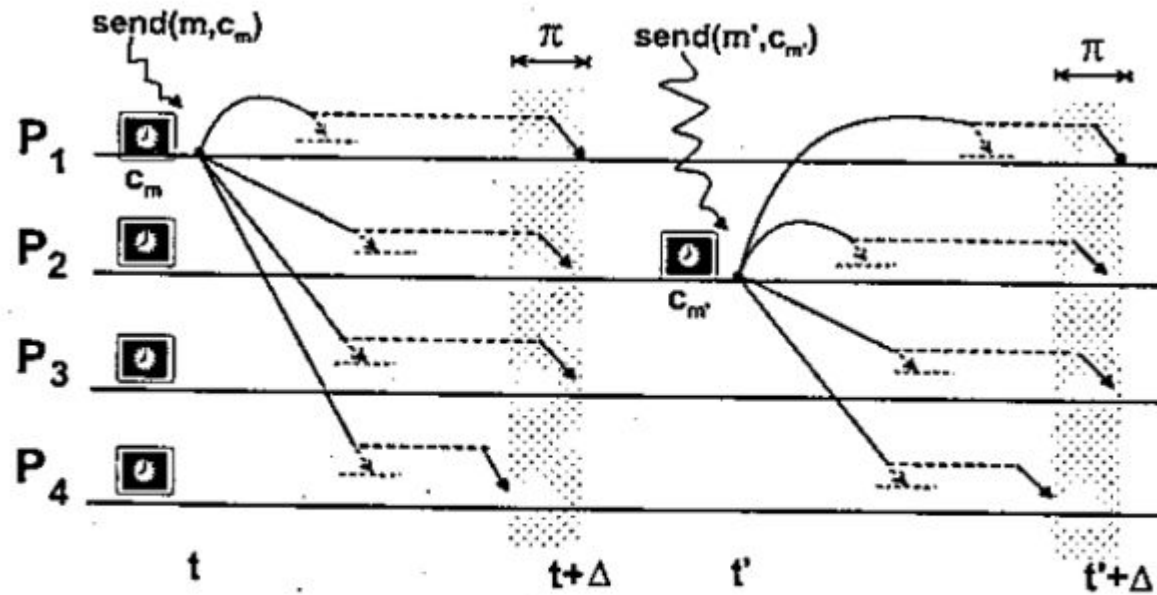


Figure 2.11. Steady and Tight Δ -protocol

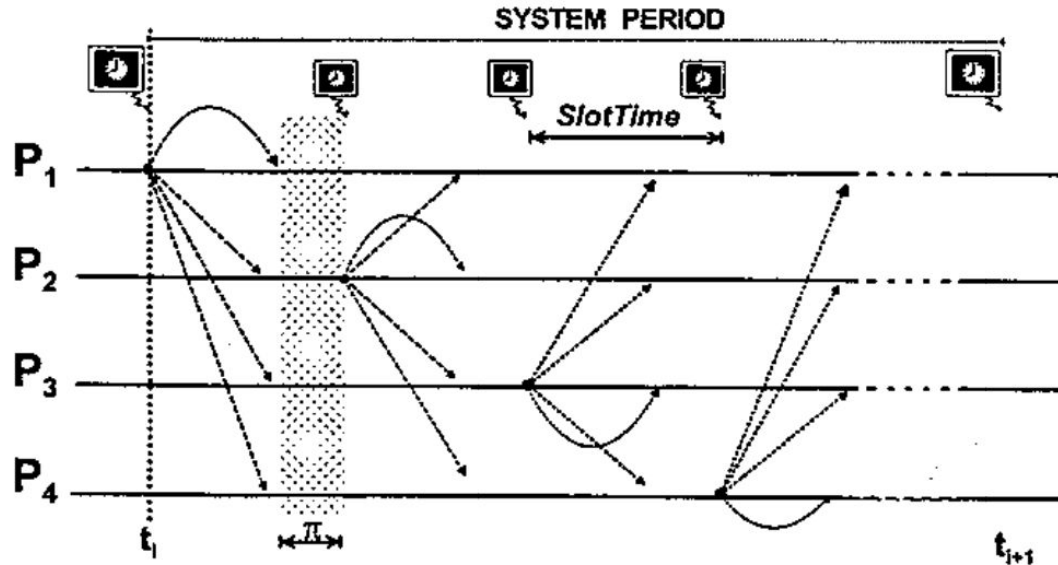


Figure 2.12. Steady and Tight TDMA Protocol

Verissimo, L. Rodriguez: Distributed Systems for Systems Architects

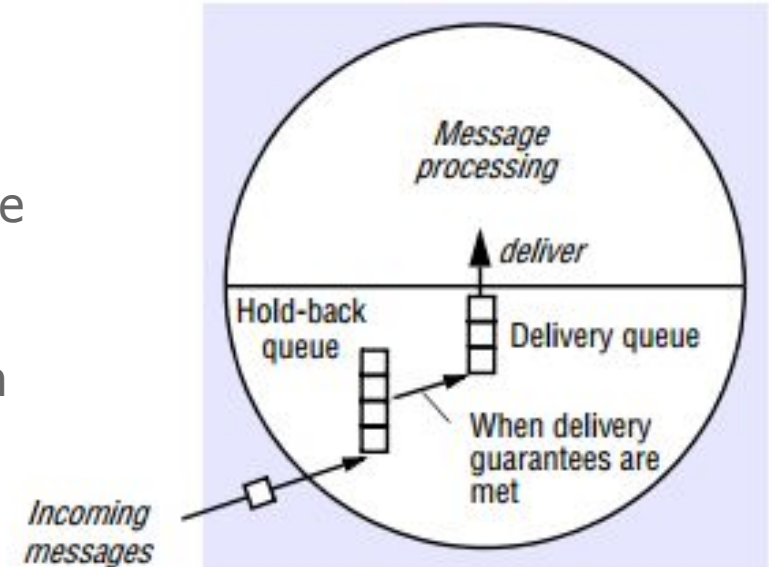


- Tiempo y relojes - Relojes Físicos
- Tiempo y relojes - Relojes Lógicos
- Sincronismo
- **Orden de Mensajes**
- Estado y consistencia



Delivery de Mensajes | Hold-back queue

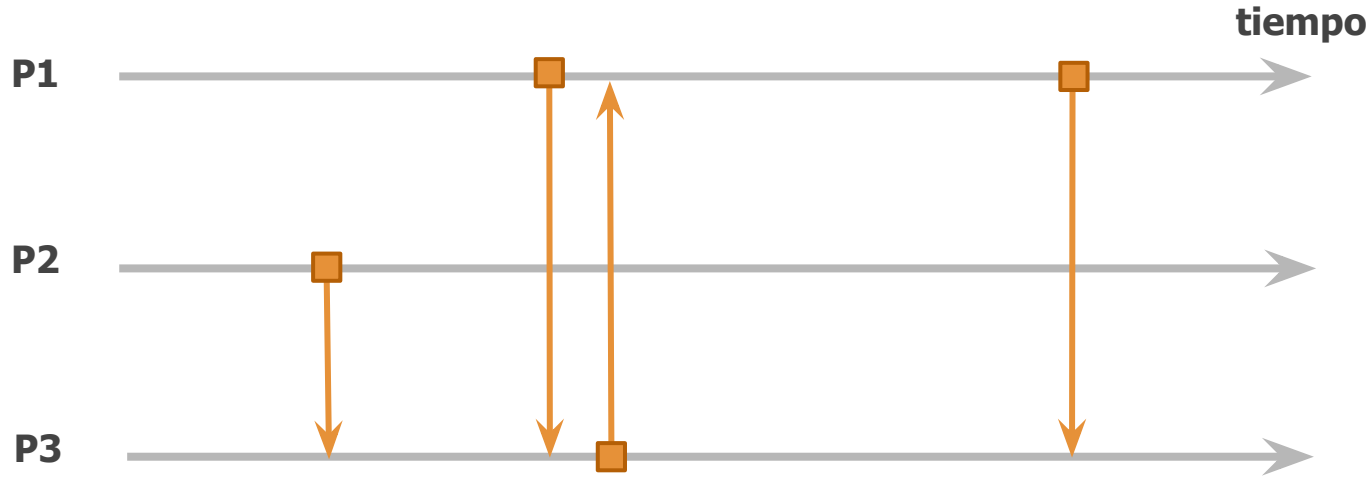
- Envío de mensajes no es lo mismo que *delivery* de los mismos
- El delivery consiste en procesar el mensaje provocando, eventualmente, cambios en el estado del proceso
- Los mensajes se mantienen en una cola que permite controlar el momento en que se lo libera, permitiendo demorar el *delivery*
- También es posible re-ordenar mensajes en esta cola





Orden Sincrónico

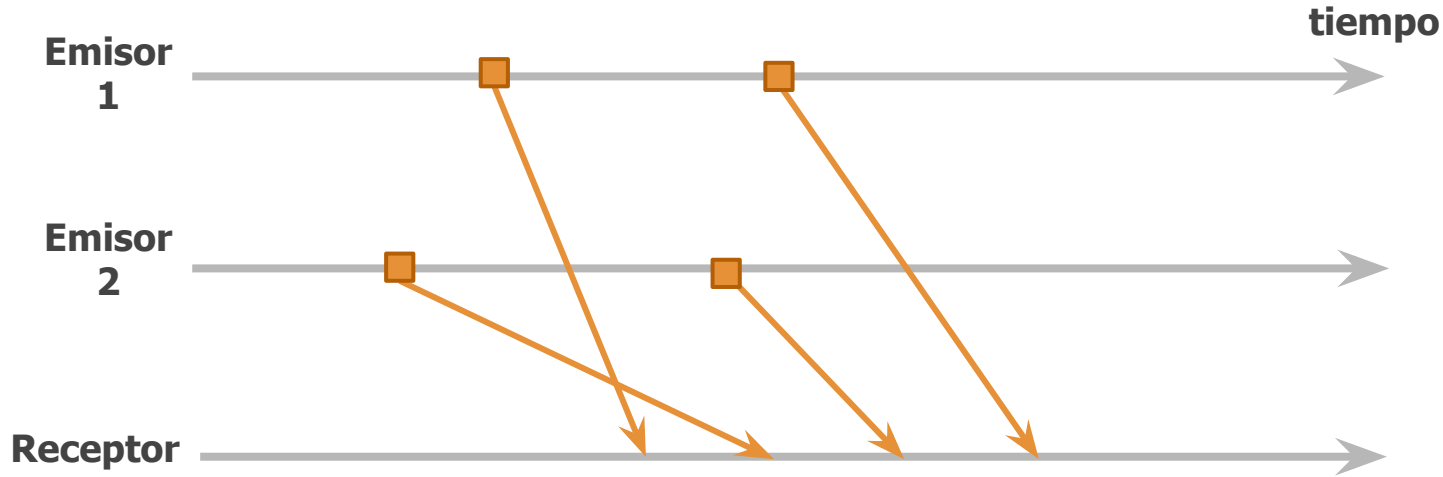
Todo mensaje posee el mismo timestamp tanto en el proceso emisor como en el receptor. Es decir, la transmisión de mensajes insume tiempo nulo.





Orden FIFO

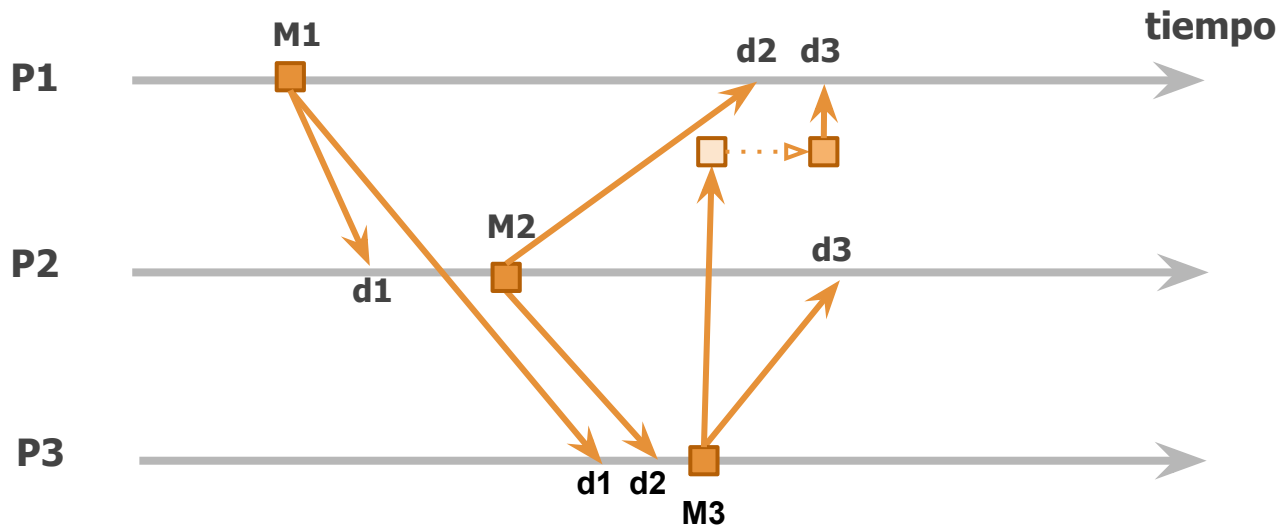
Todo par de mensajes desde un mismo emisor a un mismo receptor se entregan en el orden en que fueron enviados.





Orden Causal

Todo mensaje que implique la generación de un nuevo mensaje, es entregado manteniendo esta secuencia de causalidad sin importar el receptor.



Como:

M1 -> M2 -> M3

Luego:

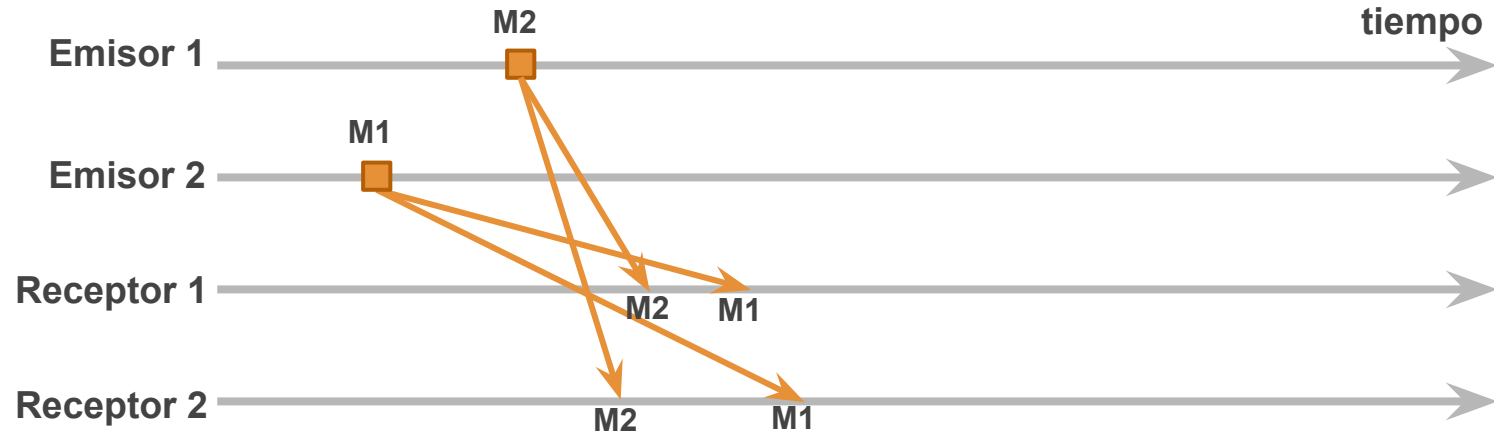
d1 -> d2 -> d3

(Nota: se asume entrega automática de cada mensaje dentro de su mismo emisor)



Orden Total

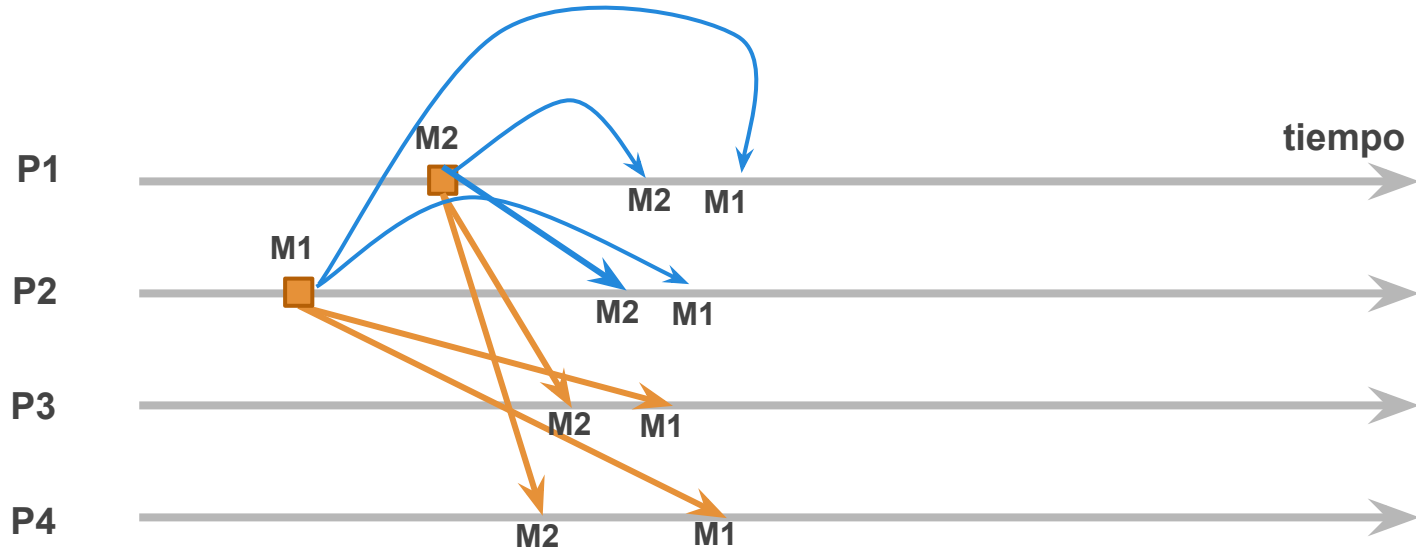
Todo par de mensajes entregado a los mismos receptores es recibido en el mismo orden por esos receptores.





Orden Total

Todo par de mensajes entregado a los mismos receptores es recibido en el mismo orden por esos receptores.



Agenda



- Tiempo y relojes - Relojes Físicos
- Tiempo y relojes - Relojes Lógicos
- Sincronismo
- Orden de Mensajes
- **Estado y consistencia**



Estado | Definiciones Local vs Global

Estado Local

Se define $\mathbf{s}_j$, el estado local del proceso j en el instante t , según

$$\mathbf{s}_j = (X_0(t), X_1(t), \dots, X_n(t))$$

con X_i = variable i del proceso j

Estado Global

Se define $\mathbf{S}$, el estado global del sistema en el instante t , según

$$\mathbf{S} = \mathbf{s}_0 \cup \mathbf{s}_1 \cup \dots \cup \mathbf{s}_n$$

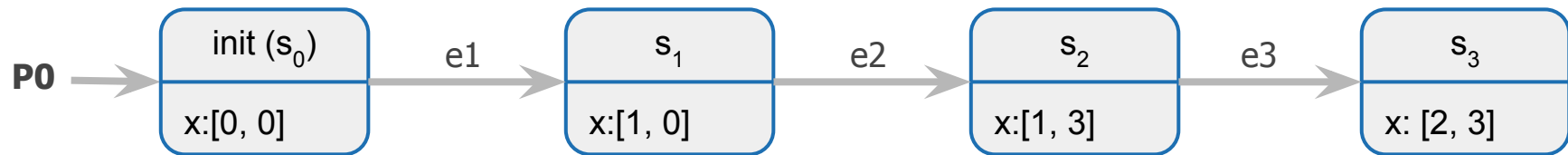
con $\mathbf{s}_i$ = estado local del proceso j



Estado | Sistema como máquina de estados

- Consiste en modelar el sistema como una serie de estados
- Un estado evoluciona al siguiente estado por la ocurrencia de un evento
- Asumiendo instrucciones determinísticas, el procesamiento de cualquier evento bajo el estado actual se puede reproducir

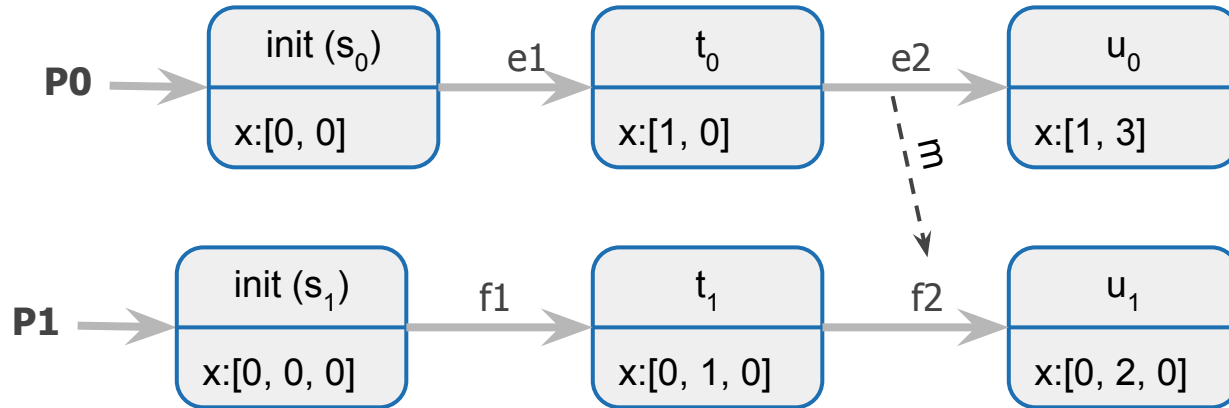
Es más simple de hacer cuando hay un único proceso. Ej:





Estado | Sistema como máquina de estados (II)

Con múltiples procesos, es más difícil determinar el estado global S. Ej.:



Estado Globales Válidos: $S = s_0 \cup s_1, t_0 \cup s_1, s_0 \cup t_1, \text{ etc}$

Estado Globales Inválidos: $S = u_1 \cup t_0$



Historia y Corte de Estados de un Sistema

Historia (o corrida)

Caracteriza al proceso P_i , considerándolo una máquina de estados, mediante la secuencia de todos los eventos procesados:

$$h_i = (e_0, e_1, e_2, \dots)$$

con e_j = todo evento aceptado por P_i

Corte

Unión del subconjunto de historias de todos los procesos del sistema hasta cierto evento k de cada proceso:

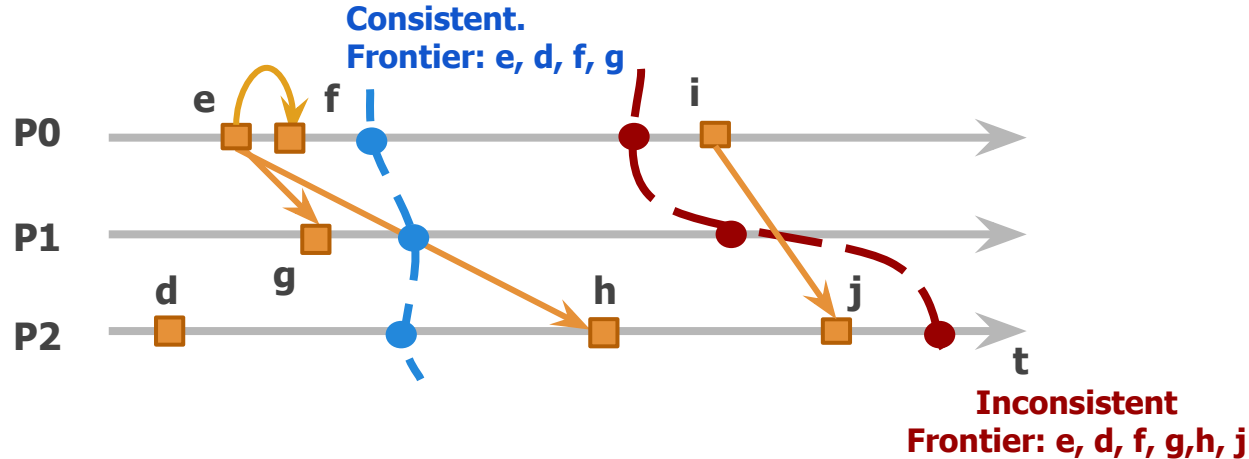
$$C = h_0 \cup \dots \cup h_n = (e_0, \dots, e_k) \cup \dots \cup (e'_0, \dots, e'_{k'})$$



Historia y Cortes | Consistencia

Un **corte** es **consistente** si por cada evento que contiene, también contiene a aquellos que 'ocurren antes' que dicho evento.

$$\forall \text{ evento } e \in C: f \rightarrow e \Rightarrow f \in C$$





Historia y Cortes | Algoritmo de Chandy & Lamport

- Algoritmo que permite obtener **snapshots** de estados globales en sistemas distribuidos
- El objetivo del algoritmo es almacenar estados de un conjunto de procesos y estados de canales (snapshots) de forma que, aunque los estados no hayan ocurrido al mismo tiempo, **el estado global almacenado sea consistente**
- **Hipótesis**
 - Los procesos y los canales de comunicación no fallan
 - Canales son unidireccionales y poseen orden FIFO
 - Grafo fuertemente conexo (camino de ida y vuelta definidos)
 - Cada proceso puede iniciar un snapshot en cualquier momento



Historia y Cortes | Algoritmo de Chandy & Lamport

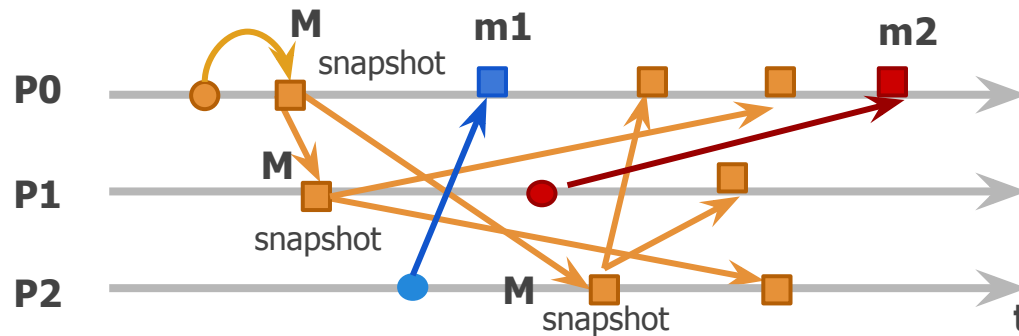
P_i
canales_out, canales_in
 $corte_i := \text{None}$
mensajes := []

Regla de envío de Marcador

Al almacenar el estado en $corte_i$
enviar Marcador por canales_out

Regla de recepción de Marcador

Al recibir un Marcador
if P_i no grabó su estado:
 $corte_i := \text{estado de } P_i$
 activar_log(mensajes, canales_in)
else:
 $corte_i += \text{mensajes}$
 mensajes := []



■ marcadores
■ mensajes

$corte = S0 \cup$
 $S1 \cup$
 $S2 \cup$
 $(P2 \rightarrow P0:m1)$



Comunicación *Reliable* | Propiedades y Orden

Comunicación *Reliable* (o Confiable): si se garantiza integridad, validez y atomicidad en el *delivery* de mensajes.

Uno a uno

- Trivial si contamos con protocolos sobre TCP/IP y una red segura

Uno a Muchos

- El grupo debe proveer las 3 propiedades: bajo TCP/IP, atomicidad requiere atención especial
- Definir el orden entre mensajes garantizado: FIFO, causal, total, etc.



Bibliografía

- Garg, V., Elements of Distributed Computing, 1st. Ed. Wiley IEEE Press, 2002.
 - Capítulo 2: Model of a computation
 - Capítulo 3: Logical Clocks
- P. Verissimo, L. Rodriguez: Distributed Systems for Systems Architects, Kluwer Academic Publishers, 2001.
 - Capítulo 2.1: Naming and addressing
 - Capítulo 2.4: Group Communication
 - Capítulo 2.5: Time and Clocks
 - Capítulo 2.6 Synchrony
 - Capítulo 2.7 Ordering
- IPSES Scientific Electronics - Standard of Time Definition
 - <https://www.ipses.com/eng/In-depth-analysis/Standard-of-time-definition>
- G. Coulouris, J. Dollimore, t. Kindberg, G. Blair: Distributed Systems. Concepts and Design, 5th Edition, Addison Wesley, 2012.
 - Capítulo 14.5 Global States
 - Capítulo 15.4 Coordination and agreement in group communication